MT CS 6210

You may talk about problems with the TA and with me, or reference any relevant literature (including the Julia documentation!), providing attribution for any good ideas you might get. You should not consult with other people or with AI tools, and your final write-up should be your own.

```
1 using LinearAlgebra
```

```
1 using Plots
```

1. Truth or dare

State whether each of the following is true or false and give a short argument why or why not.

- Suppose $r = p/q$ for $p < q$. If r can be exactly represented with k binary digits past the binary point, then it can be exactly represented with k decimal digits past the decimal point.
- If A is a square matrix with $\kappa_2(A) = \|A\|_2\|A^{-1}\|_2$ for the operator 2-norm and $r = b - A\hat{x}$, then $\|\hat{x} - x\|_2 \leq \kappa_2(A)\|r\|_2$
- A sum of two symmetric positive definite matrices must be symmetric positive definite.
- Suppose A is nonsingular and $PA = LU$ is computed via Gaussian elimination with partial pivoting (in exact arithmetic). Then all the elements of L are at most one in magnitude.

1. True. If r is exactly representable with k binary digits, its denominator divides 2^k. Thus, $r = \frac{m}{2^k}$ for some integer m. In decimal, $r = \frac{m \cdot 5^k}{10^k}$, which is exact with at most k decimal digits. for example, $\frac{1}{4} = 0.01$ (2 binary digits) = 0.25 (2 decimal digits).
2. False. The standard backward error bound for linear systems is $\|\hat{x} - x\|_2 \leq \|A^{-1}\|_2\|r\|_2$, or equivalently $\|\hat{x} - x\|_2 \leq \kappa_2(A) \cdot \|r\|_2/\|A\|_2$. The statement misses the division by $\|A\|_2$, so it overestimates the error.
3. True. The sum of symmetric matrices is symmetric. For positive definiteness, if A and B are symmetric positive definite, then for any $x \neq 0$, $x^TAx > 0$ and $x^TBx > 0$, so $x^T(A + B)x = x^TAx + x^TBx > 0$.
4. True. Partial pivoting selects the largest-magnitude entry in the current column as the pivot, so each multiplier in L (below the diagonal) is $\left|\frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}\right| \leq 1$. The diagonal of L is 1 by construction (unit lower triangular).

2. Definitely differentiable

Complete the following routine to compute the derivative of the packed LU factorization of A in the direction δA when A is strictly column diagonally dominant

```
deriv_sddlu(A, δA)
```

As a hint: Try working with $M = L^{-1}(\delta A)U^{-1}$.

You may use the following code to compute a centered finite difference estimate for comparison.

Since A is strictly column diagonally dominant, LU factorization without pivoting is stable, and A = LU where L is unit lower triangular and U is upper triangular. The packed LU stores L (lower, with 1s on diagonal) and U (upper, including diagonal) in one matrix. Differentiating A = LU gives $\delta A = \delta L U + L \delta U$, with constraints δL is strictly lower triangular (zero diagonal, since L remains unit), and δU is upper triangular. Then, $M = L^{-1}\delta A U^{-1} = L^{-1}\delta L + \delta U U^{-1}$. The strictly lower part of M gives $L^{-1}\delta L = \mathbf{tril}(M, -1)$, so $\delta L = L \cdot \mathbf{tril}(M, -1)$ (strictly lower). The upper part (including diagonal) of M gives $\delta U U^{-1} = \mathbf{triu}(M)$, so $\delta U = \mathbf{triu}(M) \cdot U$. The packed derivative is $\delta L + \delta U$

```
fd_sddlu (generic function with 1 method)
```

```julia
function fd_sddlu(A, δA; h=1e-8)
    Ap = A + h*δA
    Am = A - h*δA
    lu!(Ap)
    lu!(Am)
    (Ap-Am)/(2*h)
end
```

```
deriv_sddlu (generic function with 1 method)
```

```julia
function deriv_sddlu(A, δA)
    F = lu(A, NoPivot())
    L = F.L
    U = F.U
    M = L\(δA/U)
    δL = L*tril(M, -1)
    δU = triu(M)*U
    δpacked = δL + δU

    δpacked
end
```

3. Dratted determinant

Suppose A is well-conditioned and `F = lu(A)` is computed, and consider

$$M = \begin{bmatrix} A & b \\ c^T & d \end{bmatrix}$$

Write an $O(n^2)$ algorithm to compute the determinant of M. Your code should have the calling signature

```
result = detM(F, b, c, d)
```

Note: You may call `det(F)` for the computed LU factorization of A – that is an $O(n)$ algorithm.

This computes the determinant of the bordered matrix $M = \begin{bmatrix} F & b \\ c^T & d \end{bmatrix}$, where F is an $n \times n$ matrix, b and c are $n \times 1$ vectors, and d is a scalar. Using the block determinant formula, $\det(M) = \det(F)(d - c^T F^{-1} b)$. Solving $A^{-1}b$ via F \ b (forward substitution on L, then back substitution on U) is $O(n^2)$ as the dominant cost. The total cost is $O(n^2)$.

```
detMref (generic function with 1 method)
```

```
# Reference computation -- this is cubic time
function detMref(F, b, c, d)
    A = zeros(size(F)...)
    A[F.p,:] = F.L * F.U
    M = [A b; c' d]
    det(M)
end
```

```
detM (generic function with 1 method)
```

```
function detM(F, b, c, d)
    if size(F,1)==0
        d
    end
    detF = det(F)
    invFb = F\b
    DETm = detF*(d-dot(c, invFb))

    DETm
end
```

4. Hello, half precision!

Consider the following problem in which we are attempting to solve the same linear system in double, single, and half precision formats.

- Using the double precision result as a reference, what are the approximate normwise relative errors for the single and half precision formats?
- Using residuals (which you may compute in double precision) and the condition number estimate $\kappa(A) \approx 1.3 \times 10^4$, give estimated bounds on the normwise relative errors in each format (double, single, and half).
- Implement iterative refinement using a double-precision residual evaluation and a single-precision factorization `F = lu(As)`. Illustrate convergence of the residual.
- Iterative refinement using a double-precision residual evaluation and a half-precision factorization `F = lu(Ah)` does not converge. Why?

1. The normwise (2-norm) relative errors are Single: $\frac{||xs - xd||_2}{||xd||_2} = 1.9604e-5$, Half: $\frac{||xh - xd||_2}{||xd||_2} = 9.13595$
2. Bound for double, single, half precision are $1.44329e-12, 0.00077486, 6.34766$, resepctively

see the last 2 code blocks for part 3 and 4

3. Iterative refinement corrrects low-precision solutions using high-precision residuals. Converges to double precision because single factoes are backword stable.
4. Half-precision LU is backword unstable, and then roundoff will perturb factors significantly, so F solve for matrix A will have large relative error.

```
(
    [737.75, -38.1178, -165.008, -2128.72]
    [737.783, -38.1161, -165.036, -2128.73]
    [-1.4e4, -757.0, 1.4264e4, -2.486e3]
)
```

```
let
    A = [ 2.2721  -2.4514   2.2149   0.6596;
          0.8912  -0.9715   0.8687   0.2587;
         -1.3414   1.4472  -1.3086  -0.3894;
         -0.343    0.3701  -0.3344  -0.1]
    b =  [0.104; 0.472; 0.071; 0.895]
    As = Matrix{Float32}(A)
    bs = Vector{Float32}(b)
    Ah = Matrix{Float16}(A)
    bh = Vector{Float16}(b)

    # Solves in double, single, and half precision formats
    xd = A\b
    xs = As\bs
    xh = Ah\bh

    xd, xs, xh
end
```

```
part4 (generic function with 1 method)
```

```
function part4(A, b, xd, xs, xh)
    err_single = norm(xs - xd, 2) / norm(xd, 2)
    err_half = norm(xh - xd, 2) / norm(xd, 2)
    kappa = 1.3e4  # Given estimate
    bound_double = kappa * (eps(Float64) / 2)
    bound_single = kappa * (eps(Float32) / 2)
    bound_half = kappa * (eps(Float16) / 2)

    err_single, err_half, bound_double, bound_single, bound_half
end
```

```
(1.9604e-5, 9.13595, 1.44329e-12, 0.00077486, 6.34766)
```

```
let
    A = [ 2.2721  -2.4514   2.2149   0.6596;
          0.8912  -0.9715   0.8687   0.2587;
         -1.3414   1.4472  -1.3086  -0.3894;
         -0.343    0.3701  -0.3344  -0.1]
    b =  [0.104; 0.472; 0.071; 0.895]
    As = Matrix{Float32}(A)
    bs = Vector{Float32}(b)
    Ah = Matrix{Float16}(A)
    bh = Vector{Float16}(b)

    # Solves in double, single, and half precision formats
    xd = A\b
    xs = As\bs
    xh = Ah\bh

    xd, xs, xh
    err = part4(A, b, xd, xs, xh)
    err
end
```

```
[737.75, -38.1178, -165.008, -2128.72]
```

```
let
    A = [ 2.2721  -2.4514   2.2149   0.6596;
          0.8912  -0.9715   0.8687   0.2587;
         -1.3414   1.4472  -1.3086  -0.3894;
         -0.343    0.3701  -0.3344  -0.1]
    b =  [0.104; 0.472; 0.071; 0.895]
    As = Float32.(A)
    bs = Float32.(b)
    F = lu(As)
    x = A \ b
    for _ = 1:1000
        r = b - A * x
        delta = F \ r
        x += delta
    end
    x
end
```

```
[1420.55, -6.16539, -830.801, -2125.74]
```

```
let
    A = [ 2.2721  -2.4514   2.2149   0.6596;
          0.8912  -0.9715   0.8687   0.2587;
         -1.3414   1.4472  -1.3086  -0.3894;
         -0.343    0.3701  -0.3344  -0.1]
    b =  [0.104; 0.472; 0.071; 0.895]
    Ah = Matrix{Float16}(A)
    bh = Vector{Float16}(b)
    F = lu(Ah)
    x = A \ b
    for _ = 1:10
        r = b - A * x
        delta = F \ r
        x += delta
    end
    x
end
```

5. Pseudoinverses

Suppose $A \in \mathbb{R}^{m \times n}$ with $m > n$ is full rank. For any full rank $C \in \mathbb{R}^{m \times n}$ such that $C^T A$ is nonsingular, define $F(C)$ as the map given by solving the linear system

$$\begin{bmatrix} I & A \\ C^T & 0 \end{bmatrix} \begin{bmatrix} R \\ F \end{bmatrix} = \begin{bmatrix} I \\ 0 \end{bmatrix}$$

- Show that $F(CX) = F(C)$ for any nonsingular X, i.e. $F(C)$ really only depends on the range space of C
- Show that $F(C)$ is always a pseudoinverse for A
- Show that $F(A)$ is the Moore-Penrose pseudoinverse
- Show that if M is a symmetric positive definite matrix, $F(MA)$ is the pseudoinverse corresponding to minimizing $\phi(x) = \frac{1}{2}(b - Ax)^T M (b - Ax)$

Bonus Q: When is there an SPD M and nonsingular X so that $CX = MA$? Show how to construct such an M and X when they exist.

1. Let $C' = CX$ with $X \in \mathbb{R}^{n \times n}$ nonsingular. The system for F' is $R' = I_m - A^T F', (CX)^T R' = 0 \implies X^T C^T R' = 0$. Since X^T is nonsingular, $X^T C^T R' = 0 \implies C^T R' = 0$ (multiply by X^{-T}). Thus, $C^T(I_m - A^T F') = 0$ which is identical to the equation for $F(C)$: $C^T(I_m - A^T F) = 0$. Hence, the systems are equivalent, so $F(CX) = F(C)$. This shows $F(C)$ depends only on $\mathit{range}(C)$, as X changes the basis but not the span.
2. A pseudoinverse G satisfies $AGA = A$ and $GAG = G$. From the system, $C^T = C^T A^T F$. Since C has full column rank, $C^T A^T$ is $n \times n$ and invertible (assuming general position; principal angles nonzero). Solving $F = (C^T A^T)^{-1} C^T$. Then check $AFA = A(C^T A^T)^{-1} C^T A = A(C^T A^T)^{-1}(C^T A^T) = A$, since $C^T A = (A^T C)^T$, but computation holds by associativity. And

$FAF = (C^TA^T)^{-1}C^TA(C^TA^T)^{-1}C^T = (C^TA^T)^{-1}C^T = F$.Thus, $F(C)$ is a pseudoinverse.

3. For $C = A$, $F(A) = (A^TA)^{-1}A^T$, which is the Moore-Penrose pseudoinverse $A^\dagger$ for full column rank A (satisfies all four conditions: symmetry of $AA^\dagger$ and $A^\dagger A$).
4. The minimizer satisfies the normal equations $A^TMAx = A^TMb$, so the associated pseudoinverse is $(A^TMA)^{-1}A^TM$.
   $F(MA) = ((MA)^TA)^{-1}(MA)^T = (A^TMA)^{-1}A^TM$. This is the weighted pseudoinverse for minimizing $\min \phi(x)$

Bonus Q: This exists if and only if $range(C) = range(A)$. If condition holds: Since both subspaces are n-dimensional, there exists nonsingular $X \in \mathbb{R}^{n\times n}$ such that $C = AX$ (change of basis; $X = A^\dagger C$, and X invertible because full rank and same range). Set $M = I_m$. Then $C = I_mAX$. Construction: Compute $X = A^\dagger C = (A^TA)^{-1}A^TC$. Verify nonsingularity (det$X) \neq 0$ follows from condition). Suppose such M, X exist. M is symmetric positive definite implies invertible, so $range(C) = range(MAX) = Mrange(AX) = Mrange(A) = range(A)$ (bijective M). If ranges differ, no such M (invertible) can map one to the other while preserving dimension/full rank.